DOCUMENTATION FOR TRACKING MONITORING APP

Contents

1. System Overview

This Tracking Monitoring Application is a comprehensive solution, which has the purpose of monitoring transportation and shipment data through automated Excel files processing. The system efficiently handles data from three shipping providers (DHL, Hellmann and Logwin) transforming raw data into structured, validated information that can be easily monitored and analyzed.

The main purpose of this application is that it serves as a centralized platform for managing shipping data while offering several key capabilities:

Data Processing and Validation:

The system automates Excel file processing, implementing several validation rules to ensure data accuracy. It can process files of varying sizes, from small shipment batches to large datasets consisting of thousands of records. The validation system carefully examines each record, checking for logical consistency in dates, numerical values and minimizes data entry errors and inconsistencies.

User Interface and Visualization:

The application provides a simple and intuitive web interface with three main components. The upload page enables users to easily submit Excel files while receiving immediate feedback on the processing results. If some invalid rows are detected, the user is able to make a choice on whether to confirm the insertion of valid rows in the database, or to cancel it via confirmation dialog. The overview page presents a clear, paginated view of all tracking records while displaying essential shipping information. The detailed view offers comprehensive information about each shipment organized into a logical structure.

Provider Integration:

The application is specifically designed to handle the unique data formats of the three before-mentioned providers. It implements provider-specific mapping rules to correctly interpret and standardize data from different sources. These mapping rules are in a dedicated configuration file, which is easily expandable to handle multiple mappings if needed. This standardization ensures that regardless of the source, all shipping data is stored and displayed in a consistent format.

Error Handling and Reliability:

The system incorporates robust error handling mechanisms throughout its operation. From file upload validations to data processing and storage, each step is monitored and logged. The application provides clear, user-friendly error messages and warnings that help users quickly identify and resolve problems.

This modern, micro services-based application is built with scalability and maintainability in mind. Its architecture provides modular design, which allows for easy updates and improvements.

2. System Architecture

The application uses many modern technologies, which are chosen for their reliability, popularity, readability and scalability. These technologies are widely used so it is easier to implement this solution into greater system since it is divided into micro services, which are easily maintainable.

2.1. Frontend Technologies

- Next.js: React framework, which enables server-side rendering and simplifies routing.
- React: Framework used for building interactive user interfaces with reusable code
- Tailwind CSS: Utility-first CSS framework that ensures consistent design across the application
- TypeScript: Adds type safety to JavaScript and makes the code more reliable and, maintainable and easier to understand

2.2. Backend Technologies

- NestJS: Node.js framework which provides robust structure for building server-side applications
- TypeORM: Object-Relational Mapper that simplifies database operations and provides type safety when working with the database
- MySQL: Reliable relational database system used for storing tracking records

2.3. Development and Deployment

- Docker: Used for containerization which makes the application easy to deploy and ensures consistency across different environments
- Microservices Architecture: Application is split into separate services (file processing service, tracking service, and frontend) for better scalability and maintenance

3. Core Functionality

3.1. File Processing Service

The file processing service handles Excel file uploads, and data validation through several key components:

- File Controller handles file upload requests. It validates file types where only .xlsx and .xls are allowed. It also manages the validation and confirmation workflow.
- File Processing Service has the job of detecting the provider from the file name. It processes Excel files in chunks to handle large files more efficiently. It validates data, created detailed processing logs and manages the storage of valid records.
- File Parser reads Excel files using the xlsx library. It maps Excel columns to database fields based on the mapping tool, which is integrated in a special provider configuration file. It handles different date formats and data types and processes files in chunks to manage memory efficiently.
- Date Time Validator validates various date formats while also ensuring logical date relationships (e.g. ETD must be before ETA) and handles different date representations from Excel.

3.2. Data Validation Rules

The application implements a set of several validation checks. Date validations make sure to validate different date formats from different provider files. It also ensures the dates are logical which means that rows which have dates such as 4/38/24 or time set to 88:56:62 are immediately recognized as invalid. It also ensures that ETD occurs before ETA and it verifies logical relationships between actual and estimated time.

There are also more field validations, regardless of the date. Weight must be a positive number, packages must be a positive integer, House AWB is required and all dates must be within a reasonable range (2000-2100).

Provider-specific mapping is present in a special configuration file where each provider has specific field mappings:

- DHL: Data rows start from row 12 where header row is at row 11
- Hellmann: Data rows start from row 3 where header row is at row 2
- Logwin: Data rows start from row 1 where header row is at row 0

For the 5 given Excel files (DHL, DHL2, Hellmann, Hellman2 and Logwin), when they are uploaded, they should return a message saying:

- DHL (Total Rows: 102; Valid Rows: 66; Invalid Rows:36)

- DHL2 (Total Rows: 109; Valid Rows: 0; Invalid Rows:109 with the message: File "DHL2.xlsx" was unable to be processed. Reason: No Valid Rows.)
- Hellmann (Total Rows: 1958; Valid Rows: 772; Invalid Rows:1186)
- Hellman2 (Total Rows: 1915; Valid Rows:965; Invalid Rows:950)
- Logwin (Total Rows: 17; Valid Rows: 17; Invalid Rows: 0)

The confirmation dialog should pop up when uploading files “DHL”, “Hellmann”, “Hellmann2”. It will not pop up for “DHL2” because there are no valid rows so there is nothing to insert into the database. In addition, it will not appear for “Logwin” because there are no invalid rows.

The insertion of Hellmann files will be longer because it is a larger file. API Gateway has a timeout registered so that after 60 seconds of no response, it will return an error. If an upload of a much larger file end with a timeout, then that threshold can be changed by navigating in this file: level_up_project/services/api-gateway/src/app.module.ts and inside it there is a timeout variable set to 60000 (60 seconds). This can easily be changed to allow for a greater wait time for the response.

3.3. Log File Generation

Each successful file upload generates a detailed log file stored in the logs directory inside the file processing service folder. Every log file has the same naming pattern:

- Provider dd.mm.yyyy hh.mm.ss.log (DHL 07.01.2025 18.33.31.log)

Every log file also follows the same content structure:

- File name
- Processing Timestamp
- Total Row Count
- Valid and Invalid Row Count
- Detailed Record Information:
    - Valid records with full data
    - Invalid records with rejection reasons

These log files serve as an audit for all file processing operations.

3.4. Overview Page Pagination

The overview page implements efficient data handling through pagination. Only 50 records are displayed per page. There are navigation controls above and below the 50 records, which allow moving between pages. The table displays key information columns such as:

- Status
- PO Number
- Carrier

- Shipper
- ATA

If a records does not have the value for those columns which are on the overview page, those cells will just stay empty.

3.5. Details Page Data Display

The details page implements comprehensive data visualization. All tracking record fields are organized into logical sections:

- Shippment information
- Package Details
- Transport Details
- Time Information
- Location Information

Null values for certain fields are displayed as “—” for better readability. The source file name is displayed to trace data origin. There is also a “Back to Overview” text above the displayed information, which allows user to go back to the previous page.

Data is also formatted for enhanced readability:

- Dates in dd/mm/yyyy format
- Time beside the date is in hh:mm:ss format
- Weight with “kg” unit
- Volume with “m^3” unit

3.6. Database Structure

The database uses a single table (tracking_records) that stores all tracking-related information with fields including:

- Standard tracking fields (PO Number)
- Time-related fields (ETD, ETA, ATD, ATA and Pickup Date)
- Shipment information (Status, House AWB, Shipper Ref. No and Latest Checkpoint)
- Location information (Shipper, Shipper Country, Receiver, Receiver Country)
- Package Details (Packages, Weight, Volume)
- Transport Details (Carrier, Flight No and Incoterm)
- Audit fields (Source File, Created At, Updated At, Provider)

4. Service Communication Architecture

The communication between services is done using API Gateway so that it can ensure authenticity, and scalability in the project.

4.1. API Gateway Implementation

The API Gateway serves as the central entry point for all client requests. It manages communication between the frontend and various microservices. This implementation provides several key benefits. The gateway handles two primary types of operations:

- File Processing Operations: Managing file uploads and validation requests
- Tracking Data Operations: Forwarding tracking-related queries to the tracking service

Key features of the API Gateway include:

- Request routing to appropriate services
- Error handling and logging
- Health monitoring
- CORS configuration

4.2. Service Communication Flow

The system implements a robust communication pattern between services:

- Frontend to Gateway: The API Gateway first receives all requests from the frontend. The gateway validates incoming requests and applies necessary transformations
- Gateway to Services: File processing requests are forwarded to the File Processing Service (Port 3002). Tracking requests are forwarded to the Tracking Service (port 3003)
- Response Handling: The gateway processes service responses before sending them to the client. Errors are caught, logged and transformed into consistent response formats

4.3. Health Monitoring

The system includes comprehensive health checking capabilities. It has basic health check where the page displays basic information such as service status, environment information and timestamp of the check. However, it also has a detailed health check, which shows information that is more detailed.

This health check can be performed when a user inserts this link into the URL because there is no link on the website that navigates to that page:

- For basic health checkup: http://localhost:3001/health
- For detailed health checkup: http://localhost:3001/health/details

5. Error Handling and Logging

5.1. Centralized Error Handling

The application implements a centralized error handling system that ensures consistent error responses across all services:

- Error Categorization: Provides service-specific errors, validation errors, file processing errors and network communication errors
- Error Response Format: Consists of status code, error message, detailed information if applicable and service identifier.

5.2. Logging System

The application uses a structured logging system that provides detailed operational insights. There are three Log Levels:

- INFO: Regular operational events
- Error: Application errors and exceptions
- Debug: Detailed information for development and troubleshooting

The logs provide information about current time of the error, log level, detailed message about the error, context-specific information and stack traces for errors. There is also service-specific logging about file processing events and API gateway events. File processing events log the file size, processing duration and success status. API gateway events log request routing, service communication status and health check results.

5.3. File Processing Monitoring

The application includes specific monitoring for file processing operations. This includes performance tracking such as:

- File size monitoring
- Processing time
- Number of processed records
- Warnings for files over 150KB
- Processing status updates
- Timeout for handling large files

6. Configuration Management

6.1. Docker Configuration

The application uses Docker for database containerization, providing a consistent and isolated environment for data storage. MySQL database is configured through a Docker Compose file with the following specifications:

- Container name: levelup
- MySQL version: 8.0
- Port mapping: 3307:3306 (host: container)
- Environment settings:
    - Database name: levelup_db
    - Root password: root
    - Time zone: Europe/Belgrade
- Persistent storage through a named volume “levelup_database_dat”

6.2. Service Port Configuration

The application implements a clear port allocation for each service:

- Frontend Service: Port 3000 (default for Next.js)
- API Gateway: Port 3001
- File Processing Service: Port 3002
- Tracking Service: Port 3003
- Database: Port 3307

This port structure ensures a visible separation between services and prevents port conflicts during development and deployment.

6.3. CORS and Security Configuration

The API Gateway implements comprehensive CORS settings to ensure secure cross-origin communication.

- Allowed Methods: GET, POST, PUT, DELETE, OPTIONS, PATCH
- Credentials: Enabled for authenticated requests
- Origin: Configurable through environment variables with a safe default

6.4. Service Communication

Service endpoints are configured in the API Gateway’s configuration file (app.config.ts):

- File Processing Service URL: http://localhost:3002
- Tracking Service URL: http://localhost:3003

These endpoints are used for internal service communication, with the API Gateway acting as the central routing point for all client requests.